AI Lab assignment 11

Tejas Patil
U24AI061

Q1)

```python
def is_safe(node, color, assignment, graph):
    for neighbor in graph[node]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True


def backtrack(assignment, graph, colors):
    if len(assignment) == len(graph):
        return assignment

    unassigned = [node for node in graph if node not in assignment]
    node = unassigned[0]

    for color in colors:
        if is_safe(node, color, assignment, graph):
            assignment[node] = color

            result = backtrack(assignment, graph, colors)
            if result:
                return result

            del assignment[node]

    return None


def find_min_colors(graph):
    n = len(graph)

    for k in range(1, n + 1):
        colors = list(range(k))
```

```
        solution = backtrack({}, graph, colors)

        if solution:
            return k, solution

    return None, None



graph = {
    'Kutchh': ['Banaskantha', 'Patan', 'Surendranagar', 'Rajkot',
'Jamnagar'],

    'Banaskantha': ['Kutchh', 'Patan', 'Mehsana', 'Sabarkantha'],

    'Patan': ['Banaskantha', 'Mehsana', 'Surendranagar', 'Kutchh'],

    'Mehsana': ['Patan', 'Banaskantha', 'Sabarkantha', 'Gandhinagar',
'Ahmedabad'],

    'Sabarkantha': ['Banaskantha', 'Mehsana', 'Gandhinagar', 'Panchmahal',
'Kheda'],

    'Gandhinagar': ['Mehsana', 'Sabarkantha', 'Ahmedabad', 'Kheda'],

    'Ahmedabad': ['Mehsana', 'Gandhinagar', 'Kheda', 'Anand',
'Surendranagar', 'Bhavnagar'],

    'Kheda': ['Ahmedabad', 'Gandhinagar', 'Sabarkantha', 'Vadodara',
'Anand', 'Panchmahal'],

    'Panchmahal': ['Sabarkantha', 'Kheda', 'Vadodara', 'Dahod'],

    'Dahod': ['Panchmahal', 'Vadodara'],

    'Vadodara': ['Panchmahal', 'Dahod', 'Narmada', 'Bharuch', 'Anand',
'Kheda'],

    'Anand': ['Ahmedabad', 'Kheda', 'Vadodara', 'Bharuch'],
```

```
    'Bharuch': ['Vadodara', 'Narmada', 'Surat', 'Anand'],

    'Narmada': ['Vadodara', 'Bharuch', 'Surat'],

    'Surat': ['Bharuch', 'Narmada', 'Navsari', 'Dang'],

    'Navsari': ['Surat', 'Valsad', 'Dang'],

    'Valsad': ['Navsari', 'Dang'],

    'Dang': ['Surat', 'Navsari', 'Valsad'],

    'Bhavnagar': ['Ahmedabad', 'Surendranagar', 'Amreli', 'Rajkot'],

    'Amreli': ['Bhavnagar', 'Junagadh', 'Rajkot'],

    'Junagadh': ['Amreli', 'Rajkot', 'Porbandar'],

    'Porbandar': ['Junagadh', 'Rajkot', 'Jamnagar'],

    'Rajkot': ['Surendranagar', 'Amreli', 'Junagadh', 'Porbandar',
'Jamnagar', 'Bhavnagar', 'Kutchh'],

    'Jamnagar': ['Kutchh', 'Rajkot', 'Porbandar'],

    'Surendranagar': ['Kutchh', 'Patan', 'Ahmedabad', 'Rajkot',
'Bhavnagar']
}


min_colors, solution = find_min_colors(graph)


if solution:
    print(f"Minimum colors required: {min_colors}")
    for node in solution:
        print(f"{node} -> Color {solution[node]}")
else:
    print("No solution exists")
```

Q2)

```
def solve_send_more_money():
    solutions = []
    M = 1
    for c1 in [0, 1]:
        for c2 in [0, 1]:
            for c3 in [0, 1]:
                for S in range(1, 10):
                    if S == M:
                        continue
                    O = S + M + c3 - 10
                    if O < 0 or O > 9 or O in {S, M}:
                        continue
                    for E in range(0, 10):
                        if E in {S, M, O}:
                            continue
                        N = E + O + c2 - 10 * c3
                        if N < 0 or N > 9 or N in {S, M, O, E}:
                            continue

                        for R in range(0, 10):
                            if R in {S, M, O, E, N}:
                                continue
                            # From: N + R + c1 = E + 10*c2
                            if (N + R + c1) % 10 != E:
                                continue
                            if (N + R + c1) // 10 != c2:
                                continue
                            for D in range(0, 10):
                                if D in {S, M, O, E, N, R}:
                                    continue
                                Y = D + E - 10 * c1
                                if Y < 0 or Y > 9:
                                    continue
                                if Y in {S, M, O, E, N, R, D}:
                                    continue
                                SEND  = 1000*S + 100*E + 10*N + D
                                MORE  = 1000*M + 100*O + 10*R + E
                                MONEY = 10000*M + 1000*O + 100*N + 10*E +
Y
```

```
                                if SEND + MORE == MONEY:
                                    solutions.append((S, E, N, D, M, O, R,
Y))

    return solutions

solutions = solve_send_more_money()

for sol in solutions:
    S, E, N, D, M, O, R, Y = sol

    SEND  = 1000*S + 100*E + 10*N + D
    MORE  = 1000*M + 100*O + 10*R + E
    MONEY = 10000*M + 1000*O + 100*N + 10*E + Y

    print("Solution Found:")
    print(f"S={S}, E={E}, N={N}, D={D}, M={M}, O={O}, R={R}, Y={Y}")
    print(f"{SEND} + {MORE} = {MONEY}")
```